Relazione Progetto: Connections

Studente: Lorenzo Delitala **Corso:** Reti e Laboratorio III
A.A.: 2025/2026

---

1. Premesse e Metodologia

Per questo progetto ho deciso di adottare un approccio moderno: ho agito come **Software Architect e Senior Developer**, avvalendomi di **Gemini CLI** come assistente per la stesura materiale del codice.

Voglio essere molto trasparente su questo: ho stabilito io l'architettura, i contratti di comunicazione e le interfacce. Ho dato ordini precisi all'IA e ho revisionato ogni singola funzione per assicurarmi che la logica fosse corretta. Certo, non tutto è perfetto: ho concentrato quasi tutto il mio tempo sulla robustezza della rete e del protocollo. Questo ha fatto sì che quando sono arrivato all'implementazione interna dell'Engine e dei DAO (che risultano a tratti un po' densi e "sporchi" di dipendenze) avrei voluto rifinirli meglio, ma avevo finito il mio tempo a disposizione.

Ah, una piccola confessione: il rendering dell'output del client (colori, box, tabelle) è stato delegato quasi interamente all'IA. Odio profondamente il frontend e ho preferito che si occupasse lei di rendere "carina" la CLI mentre io mi dedicavo ai veri problemi di concorrenza sul server.

---

2. Guida alla Lettura (Il "Cuore" del Progetto)

Se dovessi dare un consiglio su come leggere il mio codice, direi di ignorare per un andare dritti alla cartella `common`, nel file `Operation.java`.

Per me, questo file è il contratto fondamentale del progetto. Siccome le specifiche JSON erano la parte più rigida della consegna, ho costruito tutto intorno a esse. In `Operation.java` ho definito ogni comando (login, register, guess, ecc.) associandolo alla sua classe di richiesta e al relativo DTO di risposta.

Ho diviso le operazioni in tre macro-categorie: 1. **User Management** (il "contorno" dell'utente) 2. **Gameplay** (la partita attiva) 3. **History & Stats** (il passato e le classifiche)

Una volta stabilito questo "contratto" tramite enum e factory method, il resto del progetto è venuto da sé come una conseguenza logica. Se capite come funziona `Operation.java`, avete capito tutto il sistema.

---

3. Architettura e Concorrenza

Il sistema è un classico **MVC distribuito**. Il Client fa da View/Controller, il Server fa da Model e gestore dello stato.

Il Client e il Pattern State

Nel client ho implementato un **Pattern State** con due stati: `LoggedOut` e `LoggedIn`. Mi sembrava assurdo lasciare che un utente potesse provare a inviare una proposta senza essere loggato. In questo modo, la logica di controllo è pulita: se non sei nello stato giusto, il comando non viene nemmeno inviato. Il thread principale gestisce l'input in modo sequenziale (aspetta l'utente -> manda richiesta -> aspetta risposta), mentre un thread separato (`NotificationService`) resta sempre vivo per "acchiappare" i pacchetti UDP del server senza bloccare la navigazione dell'utente.

Il Server e la Gestione dei Thread

La parte di cui vado più fiero è il `NioServer`. Ho implementato un ciclo `select` che non dorme mai. Ma la vera sfida è stata la concorrenza: * **Worker Pool:** Non potevo permettere che una lettura lenta su un socket bloccasse tutti gli altri. Così, il thread del Selector si occupa solo di accettare i dati e li "lancia" subito a un pool di worker. * **ConcurrentHashMap:** Ho usato queste mappe ovunque per gestire le sessioni e gli utenti online. Mi hanno salvato la vita evitandomi di impazzire con i blocchi manuali ogni volta che qualcuno si connetteva o disconnetteva. * **Sincronizzazione in PuzzleCoordinator:** Qui ho dovuto usare blocchi `synchronized` pesanti. Quando un utente invia una proposta, bisogna controllare i tentativi rimasti, calcolare i punti e aggiornare le statistiche. Queste tre cose devono succedere "tutte insieme o niente". Se due thread worker provassero ad aggiornare lo stesso utente contemporaneamente, i dati diventerebbero spazzatura. La sincronizzazione garantisce che ogni "guess" sia processata in modo atomico. Certo, si potrebbe sincronizzare sullo specifico nome dell'utente per migliorare la processione concorrente, ma l'ho ritenuto non necessario. * **Volatile per il Countdown:** Il passaggio allo stato di "attesa" tra una partita e l'altra usa un flag `volatile`. Questo assicura che quando il thread dello scheduler decide che la partita è finita, tutti i worker se ne accorgano all'istante e smettano di accettare proposte.

---

4. Persistenza e Ottimizzazioni (Espanso)

Il professore ha chiesto di gestire i file come se fossero enormi. Caricare un file da 1GB in RAM per cercare un utente non era un'opzione.

Il trucco dei Byte-Offset (PuzzlesDAO)

Per il database dei puzzle (`Connections_Data.json`), ho implementato un sistema di indicizzazione "pigro". All'avvio, il server legge il file una volta sola, ma non salva i testi. Salva solo la **posizione (l'offset in byte)** di dove inizia ogni puzzle. In questo modo, se devo caricare il puzzle numero 800, faccio una `seek()` istantanea a quel byte e leggo solo quell'oggetto JSON. È veloce, non occupa troppa memoria e scala parecchio.

La strategia Append-Only (Users e History)

Riscrivere l'intero file degli utenti ogni volta che qualcuno vince una partita sarebbe stato un suicidio in termini di performance e sicurezza dei dati. Ho scelto la strategia **Append-Only**: * Ogni "salvataggio" aggiunge semplicemente una riga in fondo al file. * All'avvio, il server legge tutto dall'inizio alla fine. L'ultima versione incontrata di un utente è quella che conta. Questo rende le scritture velocissime e, in caso di crash del server a metà scrittura, al massimo perdo l'ultimo aggiornamento, ma tutto lo storico precedente resta intatto.

JsonPersistence Utility

Per evitare di ripetere lo stesso codice in ogni DAO, ho creato una piccola utility `JsonPersistence`. Si occupa lei di configurare GSON e di assicurarsi che le cartelle esistano. È una piccolezza, ma ha reso il codice molto più pulito.

---

5. UI e Robustezza Finale

- **UDP Unicast:** Ho preferito l'Unicast al Multicast (che per il nuovo ordinamento era opzionale). Mi permette di mandare riepiloghi personali a fine partita senza che gli altri "sbircino" i risultati altrui.
- **Estetica CLI:** Ho usato i box Unicode perché vedere le parole in una griglia ordinata cambia completamente l'esperienza di gioco. I colori riflettono la difficoltà del gioco originale del NYT.
- **Graceful Shutdown:** Ho aggiunto degli Shutdown Hook sia sul server che sul client. Se premi `Ctrl+C`, il sistema non "muore" male: chiude i canali, salva i dati pendenti e, lato client, prova a fare un logout pulito per avvisare il server.

---

6. Conclusioni

Il progetto è stato un esercizio incredibile di design del protocollo. Anche se alcune parti interne del server sono venute un po' confuse per via della velocità di sviluppo, sono estremamente soddisfatto della stabilità della rete e della logica

di comunicazione. Il sistema regge bene lo stress, gestisce correttamente la persistenza e offre un'interfaccia utente che, per essere una CLI, è quasi divertente da usare.